

REPOSITORY REVIEW

# AsymptoticAnalysis after the third review wave

Mathematical contracts, Mathics portability, and the reliability of validation evidence

An incremental audit of VladimirReshetnikov/Asymptotic

## Reviewed revision

513917b5b387152256b14ac76d92dd30cd13a11d

**Review date:** September 9, 2026, America/Los\_Angeles

**Revision timestamp:** September 10, 2026, 02:16 UTC

## Abstract

This review examines the repository after its third intake of external code reviews and the introduction of substantial Mathics-specific compatibility machinery. Its principal new executable results concern the portable validation runner: an observed source change can be forgotten after restoration, and relocated modular sources can escape dependency monitoring. Both mechanisms are reproduced against the pinned upstream Python runner, with a deliberately synthetic kernel protocol; they are not claimed as Wolfram or Mathics package runs. A small patch is tested for the first defect.

The review also traces a remaining lower-Lambert-branch evaluation risk, identifies sparse-to-dense work in a Mathics algebra adapter, and characterizes a private limit-direction mismatch. Their evidence is explicitly distinguished from public-runtime reproduction. Constructive artifacts include immutable Git-source staging, exact rational enclosures for the increasing inverse of  $e^x/x$ , and hypergeometric Taylor models with negative nonintegral denominator parameters and proved tail bounds. The package is compared fairly with `Series`, `InverseSeries`, `Asymptotic`, `AsymptoticSolve`, and `DiscreteAsymptotic`. Existing review findings are used as the baseline rather than republished as discoveries.

**Evidence boundary.** Python runner experiments and independent mathematical prototypes were executed. No fresh Wolfram or Mathics execution of the package was available. Proposed Wolfram Language probes and adapters are marked unexecuted throughout.

# Contents

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive assessment</b>                                                 | <b>3</b>  |
| <b>2</b> | <b>Scope, source control, and evidence discipline</b>                       | <b>4</b>  |
| 2.1      | What was inspected and what was executed . . . . .                          | 4         |
| 2.2      | How prior findings were excluded . . . . .                                  | 4         |
| <b>3</b> | <b>Capabilities and comparison with Wolfram Language</b>                    | <b>5</b>  |
| 3.1      | The package’s mathematical object . . . . .                                 | 5         |
| 3.2      | A fair built-in comparison . . . . .                                        | 6         |
| 3.3      | Routing and observable behavior . . . . .                                   | 8         |
| <b>4</b> | <b>F1: the portable runner can forget a detected source change</b>          | <b>8</b>  |
| 4.1      | Mechanism and consequences . . . . .                                        | 8         |
| 4.2      | Executed reproduction . . . . .                                             | 9         |
| 4.3      | A narrow, tested repair . . . . .                                           | 9         |
| <b>5</b> | <b>F2: relocated modular dependencies can escape monitoring</b>             | <b>10</b> |
| 5.1      | The naming-dependent source set . . . . .                                   | 10        |
| 5.2      | Select inputs explicitly, then stage them . . . . .                         | 10        |
| <b>6</b> | <b>F3: a retained exact core can outlive an unsafe evaluator</b>            | <b>11</b> |
| 6.1      | The existing principal-branch workaround is useful but incomplete . . . . . | 11        |
| 6.2      | A sharper diagnosis: the exact SymPy conversion path . . . . .              | 12        |
| 6.3      | The concrete package path to characterize . . . . .                         | 12        |
| 6.4      | Safe containment and a bounded constructive alternative . . . . .           | 13        |
| <b>7</b> | <b>F4: resource semantics can change inside compatibility adapters</b>      | <b>14</b> |
| 7.1      | Sparse coefficient rules become a dense list . . . . .                      | 14        |
| 7.2      | A local sparse alternative, with honest limits . . . . .                    | 14        |
| 7.3      | First-position lookup has a related materialization cost . . . . .          | 15        |
| <b>8</b> | <b>F5: a private limit default is not the native default</b>                | <b>15</b> |
| <b>9</b> | <b>Mathics compatibility: what the current evidence supports</b>            | <b>16</b> |
| 9.1      | An engineered compatibility layer, not a load-only port . . . . .           | 16        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| 9.2       | Historical receipts must keep their scope . . . . .             | 18        |
| 9.3       | The next useful compatibility acceptance matrix . . . . .       | 18        |
| <b>10</b> | <b>F6: a bounded hypergeometric extension with error bounds</b> | <b>18</b> |
| 10.1      | The present admission rule is conservative . . . . .            | 18        |
| 10.2      | Coefficient recurrence and exact termination . . . . .          | 19        |
| 10.3      | A finite-radius tail theorem . . . . .                          | 20        |
| 10.4      | Transport to the package’s monomial input . . . . .             | 21        |
| <b>11</b> | <b>Development priorities and acceptance obligations</b>        | <b>21</b> |
| 11.1      | Repair evidence before expanding compatibility claims . . . . . | 21        |
| 11.2      | API clarity should follow concrete capabilities . . . . .       | 22        |
| 11.3      | Future breadth: use the existing coverage register . . . . .    | 23        |
| <b>12</b> | <b>Reproducibility and artifact guide</b>                       | <b>23</b> |
| 12.1      | What is included . . . . .                                      | 23        |
| 12.2      | Re-running the executed experiments . . . . .                   | 24        |
| 12.3      | Executing the unrun characterizations responsibly . . . . .     | 25        |
| <b>13</b> | <b>Conclusion</b>                                               | <b>25</b> |
| <b>A</b>  | <b>Minimal regression specifications</b>                        | <b>25</b> |
| <b>B</b>  | <b>Source notes and primary references</b>                      | <b>26</b> |

# 1 Executive assessment

AsymptoticAnalysis is most valuable as an explicit, inspectable calculus of selected asymptotic germs, not as a new name for ordinary power series. It combines exact real weights, complete logarithmic coefficient blocks, real-branch information, retained source models, transported remainders, several specialized scales, and separate residual, numerical, and interval-certificate facilities. These are useful distinctions even when a built-in can compute the same displayed finite expression. Its native delegation layer now preserves a separate native-result contract rather than pretending every native formal order is a proved analytic remainder.<sup>12</sup>

Three conclusions follow from the inspected revision. First, the package has a substantive implementation and an unusually detailed record of known limitations; blanket descriptions such as “only a prototype” or “does not support Mathics” would conceal important work. Second, neither complete native-function coverage nor full cross-interpreter compatibility has been established. The repository explicitly states these as goals, alongside coverage of the vendored asymptotics corpus. Third, the credibility of future compatibility claims depends on strengthening the machinery that connects test outcomes to unchanged inputs. The newly reproduced runner defects directly affect that last connection.<sup>34</sup>

| ID | Incremental result                                                                                                            | Evidence and recommended priority                                                                                           |
|----|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| F1 | A previously observed source mutation is forgotten after restoration.                                                         | Executed upstream Python runner; high priority for evidence integrity. Sticky-flag repair tested.                           |
| F2 | Dependency monitoring depends on the modular directory being named <code>Kernel</code> .                                      | Executed upstream runner with relocated-tree fixture; medium priority, independent of F1.                                   |
| F3 | Lower-Lambert exact cores retain a two-argument evaluator hazard.                                                             | Current source path plus executed independent conversion countermodel; high-impact candidate, public Mathics witness unrun. |
| F4 | A Mathics sparse coefficient adapter allocates by polynomial degree; first-position lookup also materializes all matches.     | Source-established resource costs; no claimed package benchmark or memory-exhaustion reproduction.                          |
| F5 | The private Mathics limit wrapper maps omitted direction to the left-hand limit.                                              | Source-established mismatch with the documented Wolfram default; public wrong-result reachability not established.          |
| F6 | A bounded Taylor extension can admit negative nonintegral lower parameters and terminating higher-rank hypergeometric series. | Constructive opportunity, not a claimed wrong result; rational coefficient and tail prototypes executed.                    |

Table 1: New results and carefully delimited extensions. These are not six independently reproduced package bugs.

The immediate recommendation is to repair F1, make source-set selection explicit for F2, and execute the supplied lower-branch characterization before granting numerical capability to that Mathics path. F4 and F5 call for bounded, local adapter repairs, not a rewrite of the asymptotic

<sup>1</sup>[8], public usage declarations, result construction and accessors.

<sup>2</sup>[9], native result admission and analytic-operation guard.

<sup>3</sup>[4], coverage requirements and evidence meanings.

<sup>4</sup>[6], goal, current status and pending consolidated acceptance.

engine. F6 supplies a concrete next increment in mathematical coverage with an independently checkable error contract.

## 2 Scope, source control, and evidence discipline

### 2.1 What was inspected and what was executed

The audit is pinned to commit `513917b5b387152256b14ac76d92dd30cd13a11d`, whose commit timestamp is September 10 at 02:16 UTC, or September 9 at 19:16 Pacific daylight time. The apparent date difference is a time-zone conversion, not a different revision. The packet is named `AsymptoticAnalysis`, declares version 1.8.0 and Wolfram Language 15.0 or newer.<sup>56</sup>

Inspection covered the public interface and central power-log arithmetic, ordinary inverse construction, native dispatch, selected exact-core and special-function paths, most of the Mathics bootstrap and mathematical adapters, the portable runner and selected portable tests, and the maintained review and coverage registers. The review did not execute the repository’s full suite, reproduce every historical finding, or establish that every source file is correct. In particular, source reading is not a substitute for running an evaluator-dependent Wolfram Language program.

The Wolfram execution connector returned service errors. No usable local Mathics installation was available, and attempts to obtain one did not succeed. Consequently, all fresh executable claims in this article refer either to Python infrastructure or to independent mathematics. The included Wolfram Language files are reproduction specifications and candidate code, not acceptance evidence. This distinction matters especially for F3: a defective conversion mechanism and an exposed source path are not proof that a complete public package call reaches that mechanism without an earlier refusal.

The accompanying evidence uses four categories. *Executed infrastructure* means the actual upstream Python runner was run with controlled protocol fixtures. *Executed independent oracle* means rational arithmetic, SymPy or mpmath checked an equation or modeled a conversion independently of the package. *Source-established* means the relevant current code makes the stated structural choice. *Unexecuted characterization* means a concrete Wolfram Language experiment is supplied but no outcome is asserted. Repository-provided receipts form a fifth, historical category; their counts retain their original revision and runtime scope.

### 2.2 How prior findings were excluded

The review index identifies 27 packages in three waves and 167 attributed entries before overlap consolidation. The maintained implementation register and the wave-3 crosswalk were used to screen the present findings. They are an extensive starting point, but this is not a claim to have independently reread every page of all 27 original articles or executed their supplied programs.<sup>789</sup>

---

<sup>5</sup>[1], commit identity and timestamp.

<sup>6</sup>[2], package metadata.

<sup>7</sup>[3], wave index and attributed-entry count.

<sup>8</sup>[4], complete wave-1/2 crosswalk.

<sup>9</sup>[5], complete wave-3 crosswalk.

| Item  | Nearest existing work                                                               | Specific additional contribution here                                                                                                                                                                                |
|-------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F1–F2 | V01/V02; source-freezing work in the Mathics preservation tooling                   | Current portable-runner counterexamples, including a change the runner actually detected and later forgot, plus a different relocated-dependency defect. These are not the old zero-test aggregate-runner complaint. |
| F3    | D05, X05; current Mathics documentation already warns about <code>ProductLog</code> | A lower-branch exact-core exposure trace, a sharper SymPy-conversion diagnosis, and a working rational enclosure for the original core equation. The upstream argument-order problem itself is not claimed as new.   |
| F4    | P06/P08; C01 and W3-04 concern different dense conversions                          | An additional Mathics preprocessing path with fixed support and degree-sized allocation, plus an explicitly unexecuted sparse replacement. No claim that the existing optional native export guard is absent.        |
| F5    | D06; W3-06 concerns sided Taylor germs                                              | A different private Mathics <code>Limit</code> default, with an explicit warning that changing left to right would still not reproduce the current two-sided native default.                                         |
| F6    | Existing special-function coverage and X12                                          | Concrete admissibility conditions, exact coefficient examples, and a proved finite-radius tail bound. No repetition of a generic request to “support more special functions.”                                        |

The earlier option/default inconsistencies, eager native `Normal`, observable Taylor-provider checks, refinement demand problems, and existing broad transseries proposals are not rediscovered below. Where necessary to explain the current architecture, they are identified as baseline context rather than new findings. A future merge of fixes after the pinned revision should be evaluated against that newer revision before treating any present item as still open.

### 3 Capabilities and comparison with Wolfram Language

#### 3.1 The package’s mathematical object

An ordinary power-log result is naturally read in a positive local variable  $u \rightarrow 0^+$ :

$$f(u) = \sum_{\alpha < \beta} u^\alpha P_\alpha(\log u) + \mathcal{O}\left(u^\beta (1 + |\log u|)^k\right). \quad (1)$$

The code stores a finite collection of exact weights and polynomial logarithmic coefficients, with a remainder frontier and its logarithmic degree. The local variable is a sided displacement at a finite endpoint, or an appropriate reciprocal coordinate at infinity. The public descriptor `PowerLogRemainder[u,beta,k]` is an inert mathematical remainder class, not a numerical interval.<sup>10</sup>

<sup>10</sup>[8], `PowerLogRemainder` usage, local-coordinate consumers and inverse result assembly.

Complete blocks are important. If two contributions have the same weight, their entire coefficient polynomial must be combined before determining the first omitted logarithmic degree. Conversely,  $u^\alpha$  and  $u^\gamma$  with distinct irrational weights need not fit a common rational lattice. These representation choices explain part of the package’s utility without implying that every native asymptotic expression must fit a single `SeriesData` object.

For a normalized inverse problem

$$y - y_0 = au^p \left( 1 + \sum_j u^{\delta_j} B_j(\log u) \right), \quad (2)$$

the ordinary engine works with positive gaps, an explicit real branch, and a uniformizer based on  $((y - y_0)/a)^{1/p}$ . Direct Lagrange, grouped Lagrange, Newton and incremental scheduling are existing mechanisms. The result retains its model, assumptions, source and target coordinates, coefficient blocks, and optional native view. Their existence should not be repackaged as proposed future features.<sup>1112</sup>

Other constructors deliberately leave this ordinary representation. Lambert/logarithmic inverses, Gamma and Barnes inverses, exact exponential-core corrections, finite Fourier-polynomial coefficients, and flat exponential sectors have their own small variables and truncation meanings. This is a collection of specialized, cooperating calculi; it is not yet evidence of a universal transseries algebra.<sup>131415</sup>

### 3.2 A fair built-in comparison

It would be inaccurate to compare this package only with ordinary `Series` and conclude that Wolfram Language has no asymptotic inversion. `InverseSeries` performs series reversion, including documented ramified examples. `AsymptoticSolve` supplies local asymptotic solutions of equations, including settings beyond scalar Taylor reversion. These built-ins are natural controls for overlapping admitted problems. Neither overlap nor a different output representation makes the package redundant.<sup>1617</sup>

| Task             | Built-in reference point                                                                                           | Package contribution or remaining boundary                                                                             |
|------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Local expansions | <code>Series</code> ; <code>SeriesData</code> supports a rational exponent lattice and standard series operations. | Explicit power-log blocks with exact real weights and retained analytic remainder metadata for admitted package paths. |

<sup>11</sup>[8], ordinary inverse construction, model and result fields.

<sup>12</sup>[4], P03–P08 and already implemented mechanisms.

<sup>13</sup>[10], `lambertConstruct`.

<sup>14</sup>[11], `exponentialCoreConstruct`.

<sup>15</sup>[12], Fourier coefficient and jet algebra.

<sup>16</sup>[32], scope and ramified reversion examples.

<sup>17</sup>[34], supported asymptotic equation-solving forms.

| Task                        | Built-in reference point                                                                                           | Package contribution or remaining boundary                                                                                                                 |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Series reversion            | <b>InverseSeries</b> , including Puiseux-type reversion, not merely regular linear-leading cases.                  | Several algorithms applied to explicit normalized power-log inverse models, with retained branches and source information.                                 |
| Implicit asymptotics        | <b>AsymptoticSolve</b> is a broader equation-solving interface.                                                    | Selected scalar inverse and core-perturbation families; not a replacement for every native implicit system.                                                |
| General asymptotics         | <b>Asymptotic</b> supports broad expressions and native problem forms.                                             | Specialized engines plus explicit or automatic native delegation; not complete native input coverage.                                                      |
| Discrete problems           | <b>DiscreteAsymptotic</b> handles discrete asymptotic computations, including sums and recurrence-related forms.   | No dedicated backend of that name in the inspected dispatcher. A documented coverage goal, not a newly discovered omission.                                |
| Logarithmic and exact cores | Native outputs may contain logarithms and special functions; built-in scope is not confined to <b>SeriesData</b> . | Separately recorded inverse-logarithmic variables, exact Lambert cores and complete correction blocks.                                                     |
| Arithmetic and remainders   | Built-in series have native algebra and order semantics.                                                           | Explicit propagation of package remainder descriptors and selected derivative contracts; native-only objects are not silently promoted to those contracts. |
| Numerical evidence          | Numerical equation solving and arbitrary-precision special functions are available in Wolfram Language.            | Package-specific inverse checks and selected rational interval certificates, kept distinct from symbolic residual cancellation.                            |
| Interoperability            | Native outputs retain their own heads and semantics.                                                               | <b>GeneralizedSeries</b> with <code>Kind -&gt; "Native"</code> preserves a native payload and provenance without inventing analytic exactness.             |

The native references in this comparison are the official documentation for **Series**, **SeriesData**, **Asymptotic**, and **DiscreteAsymptotic**; the package-side claims are grounded in its public declarations, native dispatcher and coverage register.<sup>181920212223</sup>

The main competitive distinction is therefore *contract visibility*. A user can ask what coordinate an error uses, whether a displayed exact core has been expanded, which source assumptions were

<sup>18</sup>[30], documented expansion forms.

<sup>19</sup>[31], representation and operations.

<sup>20</sup>[33], scope and options.

<sup>21</sup>[35], scope.

<sup>22</sup>[9], backend selector, native provenance and analytic guard.

<sup>23</sup>[7], coverage targets.

retained, and whether an operation is allowed under the stored remainder. A displayed formula alone does not answer these questions. Conversely, the built-ins have substantial breadth that the package cannot obtain merely by listing function names in an adapter or placing a native result inside another head.

### 3.3 Routing and observable behavior

The inspected dispatcher admits "Package", "Series", "Asymptotic", and Automatic. Native requests and native options can be delegated before real-coordinate admission. Automatic routing retains a successful package expansion and can search compatible native backends on admitted routes. Explicit package-only branch, direction or resource constraints are not meant to be silently discarded to make fallback succeed. Several request-resolution edge cases are already tracked by wave 3 and are not new findings in this article.<sup>2425</sup>

The native representation is a useful design decision. A computed native output, a formal order term, and a package analytic remainder theorem are different kinds of evidence. The package keeps those distinctions in metadata and rejects operations that require a package analytic contract when only a native result is available. A native Normal can also retain an infinite expression; it is not universally a finite polynomial.<sup>2627</sup>

At the ordinary object boundary, Normal[s] returns the stored expression, while numerical application substitutes a value into that expression. This is convenient, but it makes the evaluator used by a retained special-function core part of the operational contract. F3 is a concrete consequence: a mathematically correct symbolic formula can remain unsafe to specialize through a defective interpreter implementation. That is more specific than the existing general proposal for a checked numerical evaluator.

## 4 F1: the portable runner can forget a detected source change

### 4.1 Mechanism and consequences

The portable runner's purpose is appropriate: launch fresh kernels, enforce process timeouts, require a matching result protocol, retain diagnostics and reject changed-source runs. It freezes the test-suite text before executing cases. However, write\_report recomputes

```
"SourcesUnchangedDuringRun": before == after
```

on each call and overwrites the report. The final return code consults the last report's value. Nothing accumulates the fact that an earlier comparison was false.<sup>28</sup>

Let  $S_0$  be the initial tracked source state and  $S_i$  the state after case  $i$ . The intended claim requires

<sup>24</sup>[9], expansionDispatch, automaticProtectedQ and native search.

<sup>25</sup>[5], W3-01–W3-05 and W3-09.

<sup>26</sup>[9], requireAnalyticSeries and native result construction.

<sup>27</sup>[8], GeneralizedSeries usage.

<sup>28</sup>[14], fingerprints, write\_report and final return; lines 204–256.

at least

$$U_n = \bigwedge_{i=0}^n [S_i = S_0]. \quad (3)$$

The implemented final predicate is only  $[S_n = S_0]$ . Thus an  $A \rightarrow B \rightarrow A$  history can pass. The stronger point is that this is not merely a change occurring invisibly between observations: in the supplied fixture the runner observes  $B$ , writes a false unchanged flag, and subsequently erases that evidence when the source returns to  $A$ .

This can turn a mixed-source run into a success record. It does not establish that any historical repository receipt was actually contaminated; no such accusation is supported. It does establish that the present mechanism cannot justify its unchanged-during-run assertion in the reproduced scenario.

## 4.2 Executed reproduction

`code/reproduce_runner.py` copies the exact pinned upstream runner into a temporary fixture. A small executable emits the runner’s required protocol and is explicitly identified as `PROTOCOL-ONLY-FAKE-KERNEL`. One case changes a watched source; the next reads the intermediate report and restores the source. This tests the runner’s control flow and evidence handling. It does not test the asymptotic package, Mathematica, Mathics, or mathematical correctness of a test expression.

| Fixture                                | Intermediate unchanged | Final unchanged | Exit |
|----------------------------------------|------------------------|-----------------|------|
| Change, then restore; upstream         | False                  | True            | 0    |
| Change, then restore; sticky patch     | False                  | False           | 1    |
| No change; upstream                    | —                      | True            | 0    |
| No change; sticky patch                | —                      | True            | 0    |
| Permanent watched change; upstream     | —                      | False           | 1    |
| Permanent watched change; sticky patch | —                      | False           | 1    |

Table 4: Executed runner observations. See `evidence/runner_experiments.json`.

All synthetic cases report successful assertions so that source integrity is isolated from assertion failure. The unmodified runner exits zero on the restoration fixture. The repaired runner exits nonzero even though the assertions succeeded. Both implementations retain the expected behavior for unchanged and permanently changed watched inputs. These controls prevent an apparent repair that simply rejects every invocation.

## 4.3 A narrow, tested repair

The supplied patch adds a nonlocal sticky predicate:

```
sources_unchanged = True

def write_report(complete: bool) -> dict:
    nonlocal sources_unchanged
    after = fingerprints(source)
    sources_unchanged = sources_unchanged and before == after
```

```
# ...
# "SourcesUnchangedDuringRun": sources_unchanged
```

This repair is small enough to review independently. The exact diff is in `code/runner_sticky_flag.patch`; the fixture applies and executes it. A production revision should additionally retain the first detected mismatch and the affected paths after restoration. The minimal patch preserves the invalidation decision, but the upstream report’s conditional after-state field can still disappear once the files match again. A false final unchanged flag without the original mismatch details is preferable to a false success, yet it is not the most useful diagnostic.

Equation (3) still observes only snapshots. A mutation and restoration entirely between comparisons remains undetectable. The complete repair is therefore to freeze the selected source inputs as well as the suite, execute that frozen selection, and separately invalidate changes to the intended inputs or tooling. Source freezing and a sticky invalidation flag solve different problems and should not be treated as alternatives.

## 5 F2: relocated modular dependencies can escape monitoring

### 5.1 The naming-dependent source set

The same runner includes sibling Wolfram Language modules in its monitored set only when both the entry filename and parent-directory basename match a specific layout:

```
if source.name == "AsymptoticAnalysis.wl" and source.parent.name == "Kernel":
    files.update(source.parent.glob("*.wl"))
```

The package loader, by contrast, derives its module directory from the entry file’s location and uses relative `Get` calls. A copied modular package does not need the enclosing directory to be literally named `Kernel`. A path accepted for loading can therefore have a different monitored dependency set merely because its directory was renamed.<sup>2930</sup>

The executed fixture places the entry and a sibling under `RelocatedModules`, permanently changes the sibling, and emits a successful protocol result. The runner’s monitored set omits that sibling, its final unchanged flag is true, and its exit code is zero. The sibling remains changed at completion. This is independent of F1: no restoration or lost intermediate flag is needed.

The fixture’s synthetic kernel does not actually execute the sibling as Wolfram Language. That is unnecessary to demonstrate the monitoring omission, but it bounds the claim. The fact that real package loading consumes relative siblings follows from the inspected loader, not from pretending the synthetic fixture executed the package.

### 5.2 Select inputs explicitly, then stage them

A robust source contract should distinguish modular and standalone loading explicitly. For the known modular layout, select the required module set independently of the destination directory’s

<sup>29</sup>[14], `fingerprints`, lines 45–56.

<sup>30</sup>[8], `$kernelDirectory` and module-loading block.

name; for a standalone source, select the single generated file. An explicit mode or an explicit source manifest is less ambiguous than guessing from a basename. Conservatively monitoring every sibling `.wl` file is an acceptable interim policy, with the documented cost that unrelated sibling edits can invalidate a run.

The bundle provides `code/stage_git_sources.py`. It takes a repository, an immutable full commit ID, an unused output path, and a modular or standalone mode. It reads the selected bytes from Git objects rather than from the live working tree. Modular mode stages the tracked `.wl` files beneath the known `src/Kernel` layout and preserves their relative module paths. It refuses an existing destination rather than silently merging two source versions.

A synthetic local Git experiment committed `value=1`; in a helper, changed the working-tree helper to `value=999`; then staged the committed revision. The staged helper remained `value=1`; . This result is recorded in `evidence/staging_experiment.json`. It demonstrates the proposed utility’s central property; it is not a run of the Asymptotic source tree.

This utility is intentionally not a general dependency resolver, a read-only filesystem sandbox, or an arbitrary-code security boundary. A future module that reads another data file requires that file to enter the selected input set. The frozen suite, capture tooling, and source selection must be treated together. The repository’s separate definition-comparison tooling already takes source freezing seriously; the improvement here is to give the portable behavioral runner the same evidential discipline, not to claim that no frozen-input tooling exists.<sup>31</sup>

## 6 F3: a retained exact core can outlive an unsafe evaluator

### 6.1 The existing principal-branch workaround is useful but incomplete

The repository already documents a Mathics two-argument `ProductLog` problem and deliberately normalizes principal-branch construction sites to the one-argument form. It also explicitly states that numerical nonprincipal Lambert evaluation remains unsupported. This review does not rediscover that documented interpreter issue. The additional concern is how the remaining unsafe form interacts with a public result that exposes its exact core through ordinary numerical substitution.<sup>3233</sup>

The Mathics helper makes the distinction visible:

```
mathicsCoreProductLog[0, argument_] := System`ProductLog[argument];
mathicsCoreProductLog[branch_, argument_] :=
    System`ProductLog[branch, argument];
```

The principal branch is normalized, but a nonprincipal branch is forwarded to the two-argument implementation. Leaving a symbolic unsupported expression available for inspection can be legitimate. Silently applying an evaluator known to give a different function is a different matter. The generic `GeneralizedSeries` numerical accessor substitutes into the stored expression; there is no branch-specific capability refusal at that boundary in the inspected code.<sup>34</sup>

<sup>31</sup>[6], native definition comparison and frozen capture-script workflow.

<sup>32</sup>[6], Fourier and exact cores.

<sup>33</sup>[22], `mathicsCoreProductLog` and rewritten construction sites.

<sup>34</sup>[8], `GeneralizedSeries[a][val_?NumericQ]`.

## 6.2 A sharper diagnosis: the exact SymPy conversion path

The Mathics 10.0.1 `ProductLog` class declares `sympy_name = "LambertW"`. Its inherited `prepare_sympy` leaves the argument sequence unchanged, and the generic converter applies the corresponding SymPy function to those arguments. Wolfram's two-argument order is branch first; SymPy's is argument first. Thus a generic unpermuted conversion maps

$$\text{ProductLog}[k, z] \mapsto \text{sympy.LambertW}(k, z), \quad (4)$$

rather than `sympy.LambertW(z, k)`.<sup>3536</sup>

This diagnosis is narrower and more precise than saying that every bad call is caused directly by `mpmath` argument order. The inspected superclass has a separate numerical arity mechanism; no direct two-argument `mpmath` call needs to be assumed to establish the exact-conversion mismatch. The supplied independent SymPy 1.14.0 experiment evaluates both orders:

$$\begin{aligned} \text{LambertW}(-1/10, -1) &\simeq -3.577152063957297218409391963512, \\ \text{LambertW}(-1, -1/10) &\simeq -0.318131505204764135312654251588 \\ &\quad + 1.337235701430689408901162143194 i. \end{aligned}$$

The principal control is equally diagnostic: the unpermuted `LambertW(0,E)` is  $-\infty$  in that experiment, whereas `LambertW(E,0)` is 1. These are independent backend observations recorded in `evidence/mathematical_oracles.json`, not a fresh Mathics session.

## 6.3 The concrete package path to characterize

Consider

$$F(x) = \frac{e^x}{x}, \quad x > 1. \quad (5)$$

The selected increasing inverse is

$$x(y) = -W_{-1}(-1/y), \quad y > e. \quad (6)$$

The branch follows immediately from  $(-x)e^{-x} = -1/y$  and  $x > 1$ . The usual real Lambert branches are described by DLMF §4.13; the domain and identity needed here also follow directly from this equation and the derivative below.<sup>37</sup>

The package's exponential-core constructor recognizes  $av^b e^{cv^p}$  and chooses branch  $-1$  for the growing branch when  $b < 0$ . For (5), its parameters are  $a = c = p = 1$  and  $b = -1$ , so its exact core is precisely (6). When the perturbation is zero, the inspected constructor's algebra makes the expression the exact source inverse and its sector remainder zero.<sup>38</sup>

The supplied public characterization is:

```
s = AsymptoticExponentialCoreInverse[
  Exp[x]/x, 0, {x, Infinity}, {y, 0}];
Normal[s]
N[s[10], 30]
```

<sup>35</sup>[28], `ProductLog` class.

<sup>36</sup>[29], `SympyFunction.prepare_sympy`, `to_sympy`, and exact-input branch of `MPMathFunction.eval`.

<sup>37</sup>[39], defining equation and real branches.

<sup>38</sup>[11], `exponentialCoreExactInverse` and the zero-perturbation branch of `exponentialCoreConstruct`.

The correct inverse at  $y = 10$  is approximately  $3.577152063957297218409391963512$ . If the unsafe unpermuted conversion is reached, the inverse would instead be the negative of the second value above: approximately  $0.3181315052 - 1.3372357014i$ . A complete Mathics run may fail at an earlier admission or simplification step, and this review has not established otherwise. Therefore this is a *source-traced public candidate*, not a claimed observed complex package answer.

An important acceptance distinction follows. A clear unsupported-evaluator failure can be correct behavior until the numerical branch is implemented. An object that honestly retains a symbolic core can also be useful. An apparently successful numerical value for a different branch or function is not an acceptable substitute. A zero asymptotic remainder certifies the mathematical representation of an exact core; it does not certify the interpreter implementation used to evaluate that core.

## 6.4 Safe containment and a bounded constructive alternative

The immediate containment proposal is to attach numerical capability to the retained core and refuse unsupported two-argument evaluation before substitution. `code/CandidateAdapters.wl` contains a separate-context guard illustrating that boundary. It is unexecuted, not installed into the package, and not a general domain or error checker. It only screens an exposed two-argument `ProductLog`; it cannot repair an expression already corrupted by earlier evaluation. A production fix must guard construction and specialization sites that can evaluate the head before such a late screen runs.

For the real lower branch there is also a small alternative that does not require a general complex Lambert implementation. The accompanying Python code computes a rational enclosure for (5) on  $x > 1$  using only exact exponential bounds and monotonicity. It never calls Lambert  $W$  to make a proof decision.

**Lemma 1** (Rational enclosure of an exponential). *For rational  $x \geq 0$ , let  $S_n = \sum_{j=0}^n x^j/j!$  and suppose  $q = x/(n+2) < 1$ . Then*

$$S_n \leq e^x \leq S_n + \frac{x^{n+1}}{(n+1)!} \frac{1}{1-q}. \quad (7)$$

*Proof.* All omitted terms are nonnegative. Starting after the first omitted term, each successive ratio is  $x/(j+1) \leq x/(n+2) = q$ . A geometric series therefore majorizes the omitted tail. Every endpoint in (7) is rational.  $\square$

**Proposition 1** (A certified increasing inverse enclosure). *Let  $y > e$  be rational. Suppose rational  $1 \leq l < r$  satisfy a proved upper bound  $U_l < yl$  for  $e^l$  and a proved lower bound  $L_r > yr$  for  $e^r$ . Then the unique solution of (5) lies in  $(l, r)$ .*

*Proof.* The endpoint inequalities give  $F(l) < y < F(r)$ . Continuity gives a root. Moreover

$$F'(x) = \frac{e^x(x-1)}{x^2} > 0 \quad (x > 1),$$

so there is at most one root on the increasing branch. The endpoint at  $l = 1$  causes no ambiguity because  $F(1) = e < y$  and strict increase holds to its right.  $\square$

The implemented procedure verifies the left endpoint, doubles a right endpoint until its sign is proved, and bisection with rational arithmetic. When a residual sign cannot yet be decided, it tightens the exponential enclosure instead of guessing. Resource exhaustion is an explicit exception. The implementation has finite arithmetic and bracketing budgets; it is not claimed to certify every rational input within those budgets.

For  $y = 10$ , the executed code obtains

$$\frac{16892623010959913834611}{4722366482869645213696} < x(10) < \frac{67570492043839655338447}{18889465931478580854784}. \quad (8)$$

The width is  $3/18889465931478580854784 < 2^{-72}$ . The runs at  $y = 3, 10, 100$  all met the same requested width. `mpmath` values were used only as an independent numerical cross-check after the exact decisions. This is a certificate for the original core equation, not merely for a finite asymptotic model; it is not yet integrated with the Wolfram Language package.

## 7 F4: resource semantics can change inside compatibility adapters

### 7.1 Sparse coefficient rules become a dense list

The `Mathics` algebra adapter supplies a univariate `CoefficientRules` by expanding a polynomial, forming `CoefficientList`, assigning exponent indices and removing zeros.<sup>39</sup> The last filter restores a sparse *output*, but it does not undo the dense allocation already performed.

For the family

$$P_D(z) = a + bz^D, \quad D \in \mathbb{N}, \quad (9)$$

with nonzero coefficients independent of  $z$ , the natural sparse support has size two. A coefficient list has  $D+1$  positions. Thus this implementation performs at least degree-proportional list construction even though the input can encode  $D$  in  $\mathcal{O}(\log D)$  bits and the output remains two rules. At  $D = 10^6$  the dense list has 1,000,001 positions; at  $D = 10^{12}$  it would have 1,000,000,000,001. These are structural counts, not measurements of allocated memory, byte size, or a completed benchmark.

No huge polynomial was allocated to demonstrate this cost. `evidence/mathematical_oracle.s.json` explicitly records zero allocated test slots for these structural examples. A deliberate out-of-memory experiment would add little evidence to the source-level allocation argument and could obscure the distinction between a predictable complexity problem and a runtime measurement.

This path is different from the previously reviewed optional `SeriesData` export and from eager normalization of a native result. Those mechanisms have their own guards or open work. Here the intermediate coefficient provider itself becomes dense. The broad P06 resource taxonomy already exists; the new contribution is a concrete additional adapter where the taxonomy must be enforced.

### 7.2 A local sparse alternative, with honest limits

For an already expanded univariate polynomial, collect each monomial's exponent and coefficient, merge equal exponents and sort the surviving rules. The prototype `SparseCoefficientRules1` in `code/CandidateAdapters.wl` implements this idea without `CoefficientList`. Its intended behavior is

<sup>39</sup>[17], univariate `CoefficientRules` definition.

```
SparseCoefficientRules1[a + b z^D, z]
(* {{D} -> b, {0} -> a}, for positive integer D *)
```

It is an unexecuted Wolfram Language candidate, not a demonstrated Mathics repair. Its precondition excludes nonpolynomial or variable-dependent “coefficients.” Zero handling and deterministic exponent ordering are explicit; algebraic zero detection still depends on the admitted coefficient conventions.

The replacement does not make the entire package sparse. Existing core and Fourier paths also use dense coefficient lists for logarithmic polynomials, and `Expand` can itself produce excessive work before a sparse collector runs. Those are separate costs. The useful invariant is narrower: *a sparse-to-rule adapter should not allocate all absent degrees solely to recover two occupied ones.* Admission, expression expansion, coefficient normalization, and final support budgeting should each retain their own resource meaning.

For a production acceptance test, instrument the adapter’s dependency on `CoefficientList` or its allocation path, keep support fixed while increasing  $D$ , and separately measure expansion time, collection time and peak memory. Compare both output coefficients and refusal behavior. An implementation that silently drops high-degree terms to meet a budget is not an optimization of this contract.

### 7.3 First-position lookup has a related materialization cost

The Mathics compatibility helper for `FirstPosition` correctly addresses pattern matching, a lazy default and `Heads`; however, it first computes all matching positions and then takes the first.<sup>40</sup> On an expression with  $m$  matching nodes, this creates a list of all  $m$  positions for a result that needs one. This is a resource observation, not a claim that the result is mathematically wrong for ordinary pure patterns. Wolfram’s documentation itself explains first-position behavior through position-based constructions, so a stylistic objection to using `Position` would miss the actual issue.<sup>41</sup>

A bounded search is worth evaluating, either through a supported maximum-match form or an explicit traversal preserving the same level, head and ordering semantics. It must be checked on the actual Mathics version; assuming every native `Position` form is implemented would repeat the compatibility error the adapter was created to avoid. The supplied predicate-count probe is only a characterization. No timing advantage, exact predicate count, or full parity of a replacement is claimed here.

## 8 F5: a private limit default is not the native default

The Mathics limit wrapper translates explicit direction strings correctly according to the intended real-side convention: “`FromAbove`” becomes  $-1$  and “`FromBelow`” becomes  $1$ . But it also maps its default `Automatic` to  $1$ , the limit from below.<sup>42</sup>

<sup>40</sup>[16], `firstPosition` helper.

<sup>41</sup>[36], search behavior, levels and heads.

<sup>42</sup>[19], `Options[Limit]` and the direction replacement.

Current Wolfram documentation specifies `Direction -> Reals` as the default for `Limit`: the real two-sided limit. This must not be confused with either one-sided convention.<sup>4344</sup> The elementary witness is

$$h(x) = \frac{|x|}{x}, \quad \lim_{x \rightarrow 0^-} h(x) = -1, \quad \lim_{x \rightarrow 0^+} h(x) = 1.$$

There is no two-sided real limit. An omitted-direction helper that delegates to the left side asks a different question. The independent Python oracle confirms the two one-sided values; a live `Mathics` call to the package helper is supplied but unexecuted.

This is presently a private-helper contract hazard, not an established public wrong result. An inspected consumer in the native special-function adapter explicitly asks for `"FromAbove"`, and therefore avoids the default. Other consumers need an occurrence-level audit or runtime instrumentation before attributing a public failure to this helper.<sup>45</sup>

The repair must not simply change 1 to  $-1$ . That would exchange one incorrect default for another relative to Wolfram's two-sided semantics. There are two sound designs. A narrowly named positive-local-coordinate limit helper can require an explicit side and reject omission. Alternatively, a compatibility wrapper can implement a finite-real two-sided query by obtaining both one-sided results, proving equality where possible, and retaining a distinct unresolved state when the comparison cannot be established. Unequal proved finite limits justify nonexistence; two unresolved expressions do not.

Infinite endpoints and complex directions require their own semantics and must not be handled by blindly duplicating the finite real recipe. The practical first step is to make every current internal use state its intended side. That both documents the asymptotic chart and removes accidental reliance on a default whose meaning changes between runtimes.

## 9 Mathics compatibility: what the current evidence supports

### 9.1 An engineered compatibility layer, not a load-only port

The documented target is `Mathics3 10.0.1` with scanner 10.0.1, `SymPy 1.14.0` and `Python 3.11`. The repository explicitly distinguishes successful loading, selected feature examples and complete compatibility. It says consolidated acceptance of the final source remains pending. That statement should govern how its status is described.<sup>46</sup>

The inspected architecture is more substantial than a set of renamed functions. Early adapters affect how the subsequently streamed package source resolves evaluator primitives; late adapters rewrite specific private consumers where necessary. The compatibility context is removed from the user's context path after loading. The source does not replace `Mathics System` definitions for these operations; some missing `System` names are created as inert names so public syntax and package patterns agree. An inert symbol is not an implemented algorithm.<sup>4748</sup>

<sup>43</sup>[37], options and two-sided real limit examples.

<sup>44</sup>[38], direction aliases.

<sup>45</sup>[13], `specialNativeSmallQ`.

<sup>46</sup>[6], runtime target and validation boundary.

<sup>47</sup>[16], context setup, evaluator primitives and inert public names.

<sup>48</sup>[8], `Mathics`-only loading and final context-path cleanup.

The strongest parts of the design are its willingness to preserve unknown proof outcomes and its separation from Wolfram’s normal evaluation path. The assumption adapter reasons conservatively about finite real quantities and signs from explicit conjunctions rather than pretending to implement general quantifier elimination. The branch adapter supplies bounded polynomial and affine-domain facts while retaining the broader unavailable proof path as a fallback.<sup>4950</sup>

| Area                      | Inspected implementation                                                                                     | Compatibility conclusion                                                                                                             |
|---------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Evaluation control        | Private module/return translation using dynamic catch/throw tags; held inspection support.                   | Important infrastructure. Exceptional, nested and initialization cases require behavioral tests, not merely matching printed output. |
| Associations and search   | Lazy lookup defaults, key operations and pattern-aware first position.                                       | Useful bounded replacements; F4 shows that semantic repair can still change resource costs.                                          |
| Assumptions and branches  | Explicit finite-real/sign facts; selected polynomial and affine interval proofs.                             | A conservative partial proof system, not <b>Reduce</b> or <b>Resolve</b> parity. Unknown must stay unknown.                          |
| Local Taylor series       | Defining-series adapters for selected hypergeometric and polylogarithmic forms, plus scoped native fallback. | A genuine mathematical adapter with explicit restrictions; F6 gives a bounded extension.                                             |
| Gamma asymptotics         | A finite positive-argument Stirling model with a retained Poincare tail.                                     | More than load compatibility, but not a proof of all Gamma parameter regimes.                                                        |
| Lambert cores             | Principal-branch normalization to one argument at selected construction sites.                               | Useful containment; nonprincipal specialization still needs the F3 capability boundary.                                              |
| Certificates and formulas | Positive-index history update; finite-table derivative sums.                                                 | Local evaluator repairs. General numerical accuracy and certificate coverage remain feature-specific.                                |
| Time and display          | Internal proof time allowances; separate explicit user deadlines; alternative textual inspection.            | Interpreter-dependent operational policy. No measured speed ratio or notebook presentation parity is established by this audit.      |

The relevant code is in the **Mathics** compatibility, callable extraction, calculus, special-function, certificate and time-budget modules.<sup>5152535455</sup>

<sup>49</sup>[18], sign/realness facts and conservative proof outcomes.

<sup>50</sup>[21], polynomial domains, convex affine regions and eventual predicates.

<sup>51</sup>[23], held **Extract** adapter.

<sup>52</sup>[24], finite derivative-table replacements.

<sup>53</sup>[25], finite **LogGamma** Stirling adapter.

<sup>54</sup>[26], history-assignment transformation.

<sup>55</sup>[27], internal proof-time scaling.

## 9.2 Historical receipts must keep their scope

The compatibility document reports an earlier Wolfram preservation run with 1,452 passes and the same 12 pre-existing failures across 79 suites, matching 1,464 baseline per-test records. It separately describes later definition comparisons covering 2,051 modular and 2,050 standalone package symbols, selected system built-ins and behavior probes. These are repository-reported results for their identified controls and snapshots, not fresh measurements made here.<sup>56</sup>

A definition comparison is strong evidence about the captured definitions on that Wolfram runtime. It does not prove Mathics execution of those definitions, arbitrary surrounding-program equivalence, numerical accuracy, or all public option and branch combinations. Likewise, a historical full-suite comparison cannot be relabeled as a full run of later merged sources. The repository itself makes this distinction; the article preserves it.

No claim is made that F1 or F2 invalidates these existing preservation receipts. Those defects are in the portable behavioral runner inspected here. A separate frozen-source definition-comparison tool can have stronger provenance guarantees. Evidence should be attached to the exact tool and input set that produced it, not generalized from a shared directory name.

## 9.3 The next useful compatibility acceptance matrix

A consolidated run should pair modular and standalone sources with a Wolfram control and the documented Mathics environment, while recording coefficient correctness, branch/domain decisions, analytic evidence, numerical capability and operational behavior separately. A single pass/fail cell per public function hides too much: a supported symbolic constructor may have an unsupported numerical evaluator, and a correct conservative refusal may still be a coverage gap.

For the new paths in this article, the matrix can remain small and concrete. The lower-Lambert core needs principal and nonprincipal controls, symbolic construction, exact target substitution and explicit refusal behavior. The sparse collector needs zero, monomial, widely separated support, cancellation and high-degree cases. The limit wrapper needs omitted, above and below queries, a continuous control, a jump and an unresolved parameter case. The Taylor extension needs negative nonintegral denominators, lower poles, terminating rank-excess cases and genuinely divergent nonterminating controls.

In each matrix cell, distinguish successful computation, mathematically justified refusal, unresolved-within-budget outcome, interpreter exception and timeout. This is not a new proposal for a universal capability registry; it is the minimal acceptance vocabulary required by the concrete cross-runtime risks above. Complete compatibility remains a development target, not a consequence of executing these selected cells.

# 10 F6: a bounded hypergeometric extension with error bounds

## 10.1 The present admission rule is conservative

The Mathics defining-Taylor adapter accepts a constant multiple of one recognized special-function atom at a vanishing monomial argument. It verifies exact arity, independence of parameters from the

---

<sup>56</sup>[6], Wolfram preservation receipt and later native-definition comparisons.

local variable, a positive integral monomial degree, and an explicit finite order. For hypergeometric functions it also requires real upper parameters, positive lower parameters, and  $p \leq q + 1$ . A rejected defining-series request falls back to the interpreter’s native `Series` path.<sup>57</sup>

These restrictions are defensible as a first implementation. They are not all necessary mathematical restrictions. In particular, a negative nonintegral lower parameter need not cause a denominator pole, and a terminating hypergeometric polynomial does not need the nonterminating convergence-rank restriction. This section proposes a bounded extension, not a claim that a public request necessarily fails today: native simplification or fallback may already resolve individual examples.

The ordinary defining coefficients are

$$c_n = \frac{(a_1)_n \cdots (a_p)_n}{(b_1)_n \cdots (b_q)_n n!}, \quad H(z) = \sum_{n \geq 0} c_n z^n. \quad (10)$$

Excluding nonpositive integral lower parameters avoids denominator zeros. For a nonterminating defining Taylor series,  $p \leq q$  gives an entire function, while  $p = q + 1$  has unit convergence radius. A nonpositive integral upper parameter terminates the series even when  $p > q + 1$ . These standard defining-series facts supply the starting point, not the quantitative bound developed below.<sup>58</sup>

A safe first extension can be restricted to exact rational parameters. It need not solve general symbolic integer-membership questions. It should reject every lower parameter in  $\{0, -1, -2, \dots\}$  until a regularization or limiting convention is explicitly selected. Allowing upper and lower negative integers to “cancel” without specifying that convention would change the mathematical object. The accompanying prototype deliberately refuses these ambiguous cases, including a matching upper/lower value of  $-2$ .

## 10.2 Coefficient recurrence and exact termination

Starting at  $c_0 = 1$ , use

$$c_{n+1} = c_n \frac{\prod_{i=1}^p (a_i + n)}{(n+1) \prod_{j=1}^q (b_j + n)}. \quad (11)$$

Under the denominator admission rule this uses only rational arithmetic for rational parameters. If the earliest terminating upper parameter is  $-m$ , all coefficients above  $m$  are zero. Termination should be recognized before applying a convergence test appropriate only to a nonterminating series. It should also be recorded as an exact polynomial result once all coefficients through degree  $m$  are retained.

Two independently executed examples are

$${}_0F_1\left(; -\frac{1}{2}; z\right) = 1 - 2z - 2z^2 - \frac{4}{9}z^3 - \frac{2}{45}z^4 + \mathcal{O}(z^5), \quad (12)$$

$${}_3F_0(-2, 1, 1; ; z) = 1 - 2z + 4z^2. \quad (13)$$

The first uses a negative nonintegral denominator parameter; the second terminates despite exceeding the current rank gate. The coefficients follow directly from (11). They are suitable independent regression oracles, unlike copying the candidate implementation’s own result into its expected-output field.

<sup>57</sup>[20], defining-series admission and Taylor dispatch.

<sup>58</sup>[40], §16.2(i)–(iv).

### 10.3 A finite-radius tail theorem

To import a Taylor model into an analytic remainder calculus, a coefficient list alone is not the strongest available evidence. The following elementary majorant provides an explicit bound for this restricted family.

**Proposition 2** (A uniform hypergeometric Taylor-tail bound). *Assume the series in (10) is nonterminating, its lower parameters avoid nonpositive integers, and  $p \leq q + 1$ . Let  $r \geq 0$  and choose an integer  $J \geq 1$  such that  $J > \max_j |b_j|$  (taking that maximum as zero for an empty lower-parameter list) and*

$$Q_J = r J^{p-q-1} \frac{\prod_i (1 + |a_i|/J)}{\prod_j (1 - |b_j|/J)} < 1. \quad (14)$$

For truncation after degree  $m$ , put  $L = \max\{m + 1, J + 1\}$ . Then, uniformly for  $|z| \leq r$ ,

$$\left| H(z) - \sum_{n=0}^m c_n z^n \right| \leq \sum_{n=m+1}^{L-1} |c_n| r^n + \frac{|c_L| r^L}{1 - Q_J}. \quad (15)$$

For  $p \leq q$ , a sufficiently large  $J$  exists for every finite  $r$ ; for  $p = q + 1$ , one exists whenever  $r < 1$ .

*Proof.* The case  $r = 0$  is immediate. Assume  $r > 0$ . For  $n \geq J$ , the ratio of successive absolute term majorants satisfies

$$\begin{aligned} \frac{|c_{n+1}| r^{n+1}}{|c_n| r^n} &= r \frac{\prod_i |n + a_i|}{(n + 1) \prod_j |n + b_j|} \\ &\leq r n^{p-q-1} \frac{\prod_i (1 + |a_i|/n)}{\prod_j (1 - |b_j|/n)} \leq Q_J. \end{aligned}$$

The first inequality uses  $n + 1 \geq n$  and  $|n + b_j| \geq n - |b_j| > 0$ . The second uses  $p - q - 1 \leq 0$  and monotonic bounds on each factor for  $n \geq J$ . Summing the tail from  $L$  geometrically gives (15); the earlier omitted terms are a finite explicit sum. Finally  $Q_J \rightarrow 0$  for  $p \leq q$  and  $Q_J \rightarrow r$  for  $p = q + 1$ , proving the existence statements. In a terminating case the tail is instead a finite exact sum, so no ratio theorem is needed.  $\square$

The Python prototype searches a bounded sequence of  $J$  values, refuses when it cannot establish  $Q_J < 1$  within its budget, and reports the selected ratio bound. Thus the mathematical existence statement is stronger than the finite implementation's coverage, as it should be.

| Family on $ z  \leq 1/4$     | Degree retained | $Q_J$      | Tail bound |
|------------------------------|-----------------|------------|------------|
| ${}_0F_1(-1/2; z)$           | 4               | 1/2        | 1/201600   |
| ${}_3F_0(-2, 1, 1; z)$       | 2               | not needed | 0          |
| ${}_2F_1(1/3, 2/3; -1/2; z)$ | 4               | 14/27      | 77/19683   |

Table 6: Executed exact-rational coefficient and bound prototypes; not package results.

The numerical checks at  $z = \pm 1/4$  in the artifact supplement these bounds but do not prove uniformity. Uniformity comes from Proposition 2. The distinction is important: sampling two endpoints alone would not justify an error bound on a whole disk or real interval.

## 10.4 Transport to the package’s monomial input

The present adapter admits an argument  $z = ct^d$ , with fixed  $c$  and positive integral  $d$ , and an outer constant multiplier  $A$ . Choose a radius  $\rho$  with  $|c|\rho^d \leq r$ . Substituting into a proved defining-series tail gives

$$|A| \left| H(ct^d) - \sum_{n=0}^m c_n c^n t^{dn} \right| \leq |A| B_r, \quad |t| \leq \rho,$$

where  $B_r$  is the right-hand side of (15). To express a vanishing-order bound rather than just a uniform absolute number, apply the same ratio majorant after factoring  $|ct^d|^{m+1}$  from the omitted tail. This yields a computable constant times  $|t|^{d(m+1)}$  on the chosen neighborhood.

An implementation must retain the distinction between polynomial degree in  $z$  and the requested order in  $t$ . The least omitted power comes from  $d(m+1)$ , not automatically from  $m+1$ . An outer pole is not a constant multiplier and must not be admitted by this adapter without a separate order-transport argument. The current code’s constant-multiplier restriction is therefore worth preserving.

This extension can be implemented first as an internal defining-series provider, with a small explicit admission predicate and independent tests from the supplied recurrence and bounds. It should not silently broaden to divergent nonterminating  ${}_pF_q$  expansions, analytic continuation across branch cuts, regularized functions, or arbitrary symbolic parameter domains. Those are different contracts.

## 11 Development priorities and acceptance obligations

### 11.1 Repair evidence before expanding compatibility claims

The tested sticky patch should be integrated with a retained mismatch diagnostic and explicit frozen source selection. The portable runner should never turn an observed invalidation back into success, and moving a modular source tree should not change which loaded modules count as inputs. The regression fixture belongs with the runner’s own tests, not inside a symbolic package suite: it should remain executable without any Wolfram or Mathics installation.

After that, the lower-Lambert characterization should be run on both entry points and both runtimes. A failure to construct the object is different from a wrong numerical specialization and should be recorded as such. A safe refusal of unsupported numerical evaluation is a containment milestone, not full compatibility. The exact rational prototype supplies a reference value and a possible implementation route for the increasing real core, but an integrated evaluator still needs precision conventions, domain checks and resource behavior tested.

For sparse coefficient conversion, test the adapter independently before timing full inverse computations. Otherwise unrelated simplification, Fourier normalization or source parsing can mask the very allocation being repaired. The bounded first-position experiment belongs in the same resource-focused acceptance effort, with result and search-order controls preserved.

| Increment                 | Required success evidence                                                                                                     | Important negative control                                                                                              |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Portable prove-nance      | Restoration fixture remains invalid; relocated dependencies are frozen or monitored explicitly.                               | Unchanged inputs still pass; permanent changes still fail; a missing dependency cannot be silently ignored.             |
| Lower real Lambert branch | Exact-core symbolic identity and correct real specialization, or an explicit capability refusal until implemented.            | Principal branch remains correct; wrong side of the branch point and unresolved domains are not guessed.                |
| Sparse conversion         | Identical rules for a fixed-support, increasing-degree family without degree-sized coefficient allocation.                    | Dense expansion pressure and invalid polynomials still receive honest resource/admission outcomes.                      |
| Limit semantics           | Explicit side preserved; omitted finite-real query either implements two-sided behavior or is disallowed by the internal API. | A jump cannot become a two-sided value; unknown comparison is not nonexistence.                                         |
| Taylor extension          | Exact rational coefficients, termination state and a proved finite-neighborhood tail.                                         | Lower nonpositive integer poles, unselected regularization and divergent nonterminating rank-excess inputs are refused. |

## 11.2 API clarity should follow concrete capabilities

Several broad API and schema proposals are already in the repository’s review registers. This audit does not recommend starting another general registry redesign merely because a few compatibility paths need repair. The immediate missing distinction is specific: a result can have a valid symbolic core and remainder while lacking a trustworthy numerical evaluator in the current runtime. That distinction should be visible at the operation that needs it, and any metadata added for it should be exercised by F3’s acceptance cases.

Similarly, a private helper that has `Limit`’s name but intentionally uses only a positive local chart should advertise that narrower contract, rather than relying on a default that resembles native behavior but is not native behavior. For the coefficient adapter, the meaningful promise is sparse support processing within an admitted polynomial class; a nominally identical head does not guarantee identical resource costs.

Order conventions also need to be respected when comparing the package with a built-in. The package’s ordinary cutoff is exclusive in its local weight; native order specifications follow native conventions. A cutoff-two polynomial-log comparison must not be timed or judged against a native request that retains one additional Taylor power. This is a comparison methodology, not a newly discovered order-policy defect. Existing D01 and wave-3 work already address broader request normalization.<sup>596061</sup>

<sup>59</sup>[8], public cutoff conventions.

<sup>60</sup>[4], D01.

<sup>61</sup>[5], resolved-request proposals.

### 11.3 Future breadth: use the existing coverage register

The repository’s vendored-asymptotics register describes 46 articles and two reference companions and explicitly says complete coverage is not achieved. It distinguishes ordinary and  $q$ -special functions, sequence interpolation from discrete threshold recovery, fixed parameters from uniform regimes, and proved statements from formal, conditional or conjectural ones. Those distinctions should remain the development plan rather than be replaced by a new undifferentiated list of missing functions.<sup>62</sup>

The constructive increments here fit that plan without claiming to solve it. The real lower-Lambert enclosure strengthens a reusable exact-core evaluation path. The hypergeometric provider supplies a broader but still proved local analytic family. Sparse adapters reduce a specific risk that can recur as larger logarithmic or special-function coefficient families are admitted. None of these establishes complete  $q$ -asymptotic, saddle-point, arithmetic-sector, complex-sector or multivariate coverage.

A useful future acceptance record should pair each new algorithm with the source definition, chosen branch or interpolation, parameter regime, coefficient oracle and error theorem it implements. Executing finitely many orders supports an implementation claim; an all-orders claim additionally requires an argument that its recurrence and truncation rule preserve the invariant. The proofs and bounded algorithms in this article are intended as examples of that narrower, verifiable progression.

## 12 Reproducibility and artifact guide

### 12.1 What is included

The archive contains the article source and PDF, the exact pinned portable runner excerpt with its upstream license, seven runner regression fixtures, the tested sticky patch, immutable Git-source staging, independent rational and symbolic oracles, recorded observations, and unexecuted Wolfram Language characterizations and candidates. It contains no checksum files. The upstream runner itself computes source digests as part of its ordinary operation; retaining that source code is not the inclusion of a checksum artifact.

| Artifact                                   | Status and purpose                                                                                                                                                 |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>code/reproduce_runner.py</code>      | Executed. Runs the actual upstream Python runner through seven synthetic protocol fixtures and tests the sticky patch.                                             |
| <code>code/runner_sticky_flag.patch</code> | Executed in the fixture. Narrow F1 repair only; does not fix F2 or unobserved transient edits.                                                                     |
| <code>code/stage_git_sources.py</code>     | Executed on a synthetic Git repository. Stages the known source layout from a full immutable revision; <code>code/reproduce_staging.py</code> reruns that fixture. |

---

<sup>62</sup>[7], corpus, status and article-to-computation requirements.

| Artifact                                            | Status and purpose                                                                                                                                                   |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>code/exact_oracles.py</code>                  | Executed. Rational exponential-core enclosures, hypergeometric coefficients and tails, explicit one-sided limits, and the independent SymPy conversion countermodel. |
| <code>code/TargetedProbes.wl</code>                 | Unexecuted. Concrete package and private-helper characterization requests. Not an automatic acceptance suite.                                                        |
| <code>code/CandidateAdapters.wl</code>              | Unexecuted. Separate-context sparse collector and numerical capability guard; no package or <code>System</code> definitions changed.                                 |
| <code>evidence/runner_experiments.json</code>       | Selected actual runner observations. Synthetic runtime labels are retained. Timing here is not package performance.                                                  |
| <code>evidence/mathematical_oracles.json</code>     | Executed exact arithmetic results and independent numerical checks, clearly labeled as not package output.                                                           |
| <code>evidence/staging_experiment.json</code>       | Executed immutable-staging observation from a synthetic local Git fixture.                                                                                           |
| <code>upstream-excerpts/run_mathics_tests.py</code> | Pinned upstream source used by the runner experiments, accompanied by the original license.                                                                          |

## 12.2 Re-running the executed experiments

The runner fixture uses only Python’s standard library and a POSIX executable fixture; Linux, macOS or WSL is appropriate. It launches no genuine kernel. From the extracted archive directory:

```
python code/reproduce_runner.py
python code/exact_oracles.py
```

The mathematical oracle script was executed with Python 3.13.5, SymPy 1.14.0 and mpmath 1.3.0. The Mathics project’s documented Python 3.11 environment is a different environment and was not exercised by these Python-only checks. The rational enclosure decisions do not depend on floating-point rounding; SymPy/mpmath are used for the independent conversion and numerical controls.

The staging utility requires a local Git repository that actually contains the selected commit. Its modular example is:

```
python code/stage_git_sources.py /path/to/Asymptotic \
513917b5b387152256b14ac76d92dd30cd13a11d \
/path/to/new-frozen-snapshot --mode modular
```

An existing destination is refused. This command performs local source staging; it does not modify the repository or run a test suite. A separate invocation can stage the generated standalone source with `-mode standalone`.

### 12.3 Executing the unrun characterizations responsibly

For an actual Mathics characterization, create the repository’s documented environment, stage the pinned source, and set `ASYMPTOTIC_AUDIT_SOURCE` to the entry file. Stream `code/TargetedProbes.wl` as a file, rather than putting `Get` and previously unknown package names in a single pre-parsed command string. The repository documents why the latter can bind names in the wrong context.<sup>63</sup>

Use an operating-system process timeout and preserve the complete output, including failed construction, messages and exceptions. The supplied probe file prints observations; it deliberately does not convert every unsupported result into a pass. Its candidate expected values are grounded in the independent mathematics above. An actual acceptance suite should isolate cases in fresh processes, capture held outputs, and compare semantic results without discarding branch or remainder information.

The candidate Wolfram Language adapter file is not automatically loaded by the probe file. This avoids accidentally testing a modified environment while labeling it the upstream baseline. Baseline and candidate evaluations should be separate, and a repair should be claimed only after both the positive witness and its controls have been run. This article’s archive does not change any GitHub file or commit any patch.

## 13 Conclusion

The inspected revision has clear mathematical strengths: specialized exact-weight calculi, explicit real branches, retained precision information, exact cores and a native-result contract that avoids conflating formal output with analytic evidence. Its Mathics work is substantial and deliberately conservative, but complete compatibility remains unproved and explicitly unfinished in the repository’s own records.

The strongest new findings are not speculative algebraic counterexamples. They are two reproduced defects in the current portable runner’s connection between observations and source identity. Repairing that connection is a prerequisite for trustworthy compatibility claims. The other new results are deliberately narrower: a lower-Lambert specialization risk traced to an exact conversion path, a degree-sized allocation inside a sparse-looking Mathics adapter, and a private limit default that asks a different question from the current native default.

The accompanying code moves beyond criticism without overstating completion. One provenance repair is tested against the actual runner. Immutable source staging is demonstrated independently. The original exponential-core equation receives exact rational enclosures, and a bounded hypergeometric extension receives explicit coefficients and a proved tail majorant. The remaining Wolfram and Mathics experiments are supplied as concrete, unexecuted acceptance work. These results add to the existing review program without relabeling its backlog as new discoveries.

## A Minimal regression specifications

**F1 invariant.** Once a source comparison is false, every later report from that invocation must remain invalid, even if all watched files are restored byte-for-byte. Preserve the first mismatch

---

<sup>63</sup>[6], installation and streaming-load notes.

details. An unchanged run must remain admissible.

**F2 invariant.** Renaming a correctly relocated modular directory must not remove a loaded sibling from the selected source set. A standalone entry must remain distinguishable from modular loading without depending solely on the destination basename.

**F3 invariant.** A mathematically admitted lower-real Lambert core must either specialize through a tested branch-correct evaluator or return an explicit unavailable-capability result. A high-precision number for the wrong function cannot count as compatibility. Both principal and lower-branch controls are required.

**F4 invariant.** For an admitted expanded polynomial with fixed support, coefficient-rule collection must not allocate every absent degree merely to filter them out. Preserve zero, coefficient and ordering semantics. Bound expansion separately.

**F5 invariant.** A finite real two-sided query cannot be satisfied by one side alone. Explicit local-side requests must be preserved. Unknown one-sided information or unproved equality remains unresolved rather than guessed.

**F6 invariant.** Lower denominator poles are refused unless a different mathematical convention is explicitly selected. A terminating series can bypass the nonterminating rank condition, but exactness is asserted only when the retained finite polynomial is complete. Every nonterminating analytic import retains a justified tail.

## B Source notes and primary references

Repository references below are pinned to the reviewed revision unless explicitly identified as upstream Mathics tag 10.0.1. Wolfram and NIST references are live primary documentation consulted for the review; they are not evidence that a corresponding package call was executed. Code locations are identified by source file and function in the footnotes. The existing review registers supply the novelty baseline; this article’s JSON observations supply its fresh executable evidence.

## References

- [1] Asymptotic contributors. *Reviewed commit*.  
<https://github.com/VladimirReshetnikov/Asymptotic/commit/513917b5b387152256b14ac76d92dd30cd13a11d>.
- [2] Asymptotic contributors. *Paclet metadata*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/PacletInfo.wl>.
- [3] Asymptotic contributors. *External code-review index*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/external-reports/code-review/README.md>.
- [4] Asymptotic contributors. *Implementation status and complete wave-1/2 finding crosswalk*.

- [https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/development/CODE\\_REVIEW\\_STATUS.md](https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/development/CODE_REVIEW_STATUS.md).
- [5] Asymptotic contributors. *Wave-3 intake and complete finding crosswalk*.  
[https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/development/WAVE\\_3\\_INTAKE.md](https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/development/WAVE_3_INTAKE.md).
- [6] Asymptotic contributors. *Mathics compatibility: goals, adapters and evidence boundary*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/Mathics/COMPATIBILITY.md>.
- [7] Asymptotic contributors. *Vendored asymptotics: implementation coverage matrix*.  
[https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/development/VENDORED\\_ASYMPTOTICS.md](https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/docs/development/VENDORED_ASYMPTOTICS.md).
- [8] Asymptotic contributors. *Core package: public contracts, arithmetic, inverse construction and loading*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/AsymptoticAnalysis.wl>.
- [9] Asymptotic contributors. *Native delegation and result contracts*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/NativeCompatibility.wl>.
- [10] Asymptotic contributors. *Lambert and logarithmic inverse construction*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/LambertInverse.wl>.
- [11] Asymptotic contributors. *Exact exponential-core perturbation inverses*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/ExponentialCorePerturbation.wl>.
- [12] Asymptotic contributors. *Finite Fourier-polynomial coefficient algebra*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/FourierCoefficients.wl>.
- [13] Asymptotic contributors. *Native special-function asymptotic import*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/NativeSpecialFunctions.wl>.
- [14] Asymptotic contributors. *Portable Mathics/Wolfram test runner*.  
[https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/validation/run\\_mathics\\_tests.py](https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/validation/run_mathics_tests.py).
- [15] Asymptotic contributors. *Selected portable exact regression definitions*.  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/validation/MathicsTests.wl>.

- [16] Asymptotic contributors. *Private Mathics evaluator compatibility helpers.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsCompatibility.wl>.
- [17] Asymptotic contributors. *Private Mathics algebra helpers.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsAlgebra.wl>.
- [18] Asymptotic contributors. *Conservative Mathics realness and sign proofs.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsAssumptions.wl>.
- [19] Asymptotic contributors. *Mathics simplification, Taylor and limit wrappers.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsSimplification.wl>.
- [20] Asymptotic contributors. *Mathics defining Taylor-series provider.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsTaylor.wl>.
- [21] Asymptotic contributors. *Mathics polynomial and affine branch proofs.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsInverseBranches.wl>.
- [22] Asymptotic contributors. *Mathics exact-core special-function normalization.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsCoreFunctions.wl>.
- [23] Asymptotic contributors. *Mathics held callable extraction.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsCalls.wl>.
- [24] Asymptotic contributors. *Mathics finite derivative-table replacements.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsCalculus.wl>.
- [25] Asymptotic contributors. *Mathics finite Stirling-model adapter.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsSpecialFunctions.wl>.
- [26] Asymptotic contributors. *Mathics certificate history-assignment adapter.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsCertificate.wl>.
- [27] Asymptotic contributors. *Mathics internal proof time allowances.*  
<https://github.com/VladimirReshetnikov/Asymptotic/blob/513917b5b387152256b14ac76d92dd30cd13a11d/src/Kernel/MathicsTimeBudget.wl>.

- [28] Mathics3 contributors. *ProductLog implementation, tag 10.0.1*.  
<https://github.com/Mathics3/mathics-core/blob/10.0.1/mathics/builtin/specialfunctions/expintegral.py>.
- [29] Mathics3 contributors. *Generic SymPy conversion and exact-input evaluation, tag 10.0.1*.  
<https://github.com/Mathics3/mathics-core/blob/10.0.1/mathics/core/builtin.py>.
- [30] Wolfram Research. *Wolfram Language documentation: Series*.  
<https://reference.wolfram.com/language/ref/Series.html>.
- [31] Wolfram Research. *Wolfram Language documentation: SeriesData*.  
<https://reference.wolfram.com/language/ref/SeriesData.html>.
- [32] Wolfram Research. *Wolfram Language documentation: InverseSeries*.  
<https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [33] Wolfram Research. *Wolfram Language documentation: Asymptotic*.  
<https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [34] Wolfram Research. *Wolfram Language documentation: AsymptoticSolve*.  
<https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [35] Wolfram Research. *Wolfram Language documentation: DiscreteAsymptotic*.  
<https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>.
- [36] Wolfram Research. *Wolfram Language documentation: FirstPosition*.  
<https://reference.wolfram.com/language/ref/FirstPosition.html>.
- [37] Wolfram Research. *Wolfram Language documentation: Limit*.  
<https://reference.wolfram.com/language/ref/Limit.html>.
- [38] Wolfram Research. *Wolfram Language documentation: Direction*.  
<https://reference.wolfram.com/language/ref/Direction.html>.
- [39] NIST Digital Library of Mathematical Functions. *Section 4.13: Lambert W-function*.  
<https://dlmf.nist.gov/4.13>.
- [40] NIST Digital Library of Mathematical Functions. *Section 16.2: Generalized hypergeometric functions, definition and analytic properties*.  
<https://dlmf.nist.gov/16.2>.